SPECIFICATION FOR QUOTATION

Mosquito Alert mobile app: dependency and toolchain upgrade

One package of work on the Mosquito Alert Flutter app for Android and iOS: bring Flutter, the build tooling and every package up to date in a single verified sweep, with no change in what the app does and no loss of users' accounts or data when they update.

CLIENT

Mosquito Alert · Universitat Pompeu Fabra (UPF)

CONTACT

John Palmer · john.palmer@upf.edu · it@mosquitoalert.com

QUOTATIONS DUE

Tue 17 Nov 2026, 12:00 CET (§10)

WORK PERIOD

Kick-off Fri 20 Nov 2026 · delivery by Fri 15 Jan 2027 at the latest

PLATFORM	SCOPE	CODEBASE	BACKEND	REQUIREMENTS
Flutter · Android and iOS · GPL-3.0	One package · one contractor	13,500 lines · 55 packages, 39 behind	Out of scope · SDK unchanged	26 must · 2 should

CONTENTS

1 Summary
2 Background
3 Timeline
4 Scope
5 The app today
6 Requirements
7 Working rules (TECH)
8 What the project team provides
9 Deliverables and acceptance
10 How to submit a quotation
A Versions today
B What to check on each device
C Environments, identities and secrets

1 Summary

The Mosquito Alert mobile app, for Android and iOS, is a Flutter app maintained by a small research team. Its Flutter version, its build tooling and most of its packages have fallen behind: 39 of its 55 direct packages have newer releases, ten of them a new major version. We want a contractor to **bring the whole app up to date in one verified sweep, with no change in what the app does**, so that the team can build on a current base and keep it current afterwards.

This is one package of work for one contractor. It is delivered as pull requests to the public repository. It is finished when the app builds on the current stable Flutter with every dependency current or held back for a stated reason, when a user who updates from the version in the stores keeps their account and their data, and when this has been checked on physical devices. The method is yours: section 6 says what must be true at the end, not how to get there.

A trial run suggests that the Dart side is small and that the Android build needs care (Appendix A.5). On the current Flutter the existing tests pass unchanged, and taking every package to its latest version produces two compile errors in the app's code, both from one package, and one incompatibility between two packages. The Android build then fails until the compile SDK and the Android build tooling are moved together. We expect most of the effort to go into the Android and iOS builds and into verifying behaviour on devices. On Android this includes the change that every build warns about today: plugins that apply the Kotlin Gradle plugin, which a future Flutter will refuse to build (UPG-05).

Quotations are due by **Tuesday 17 November 2026, 12:00 CET** (section 10). Work starts with a kick-off on **Friday 20 November** and is delivered by **Friday 15 January 2027** at the latest (section 3).

Each requirement has one priority: **MUST** (required) or **SHOULD** (expected; tell us if dropping it would lower the price). Requirement IDs (e.g. `UPG-03`) are there so that you can refer to them in your quotation.

2 Background

Mosquito Alert is a citizen-science project that tracks disease-carrying mosquitoes with the help of volunteers, who report mosquitoes, bites and breeding sites with the mobile app. The reports, with their photos, are validated by entomologists and used by public-health authorities and researchers. The project is coordinated by research centres in Catalonia (CEAB-CSIC, UPF, CREAF and ICREA) and works with partners across Europe and beyond.

The app has been in the stores since 2014. The current codebase is a Flutter app that talks to the project's REST API through a generated Dart SDK, uses Firebase for push notifications and analytics, Google Maps for locating reports, the device camera for photos, and background tasks for sending reports saved offline and for optional location sampling. It is translated into 26 languages. In 2026 the team added Arabic, the app's first right-to-left language, and split off a test app with its own identity, backend and store listings, so that testers can file reports without polluting the production data.

The app was last brought up to date in mid-2026, when it moved to Flutter 3.44. Since then the team's time has gone into release 4.4.0, and updates have accumulated. Dependabot proposes them every month, each one needs the same manual check on devices, and nine of its pull requests are open. Doing them together, once, with a proper check, costs less than doing them one by one, and we want it done before the next round of feature work starts from this code.

The team is small, and product decisions, translations and store releases stay in house. What we want from a contractor is well-bounded engineering work, delivered as reviewable pull requests into a codebase that we continue to maintain.

3 Timeline

DATE	STEP
Until Mon 16 Nov 2026, 12:00 CET	Questions by email to john.palmer@upf.edu, copying it@mosquitoalert.com. Answers are shared with all bidders.
Tue 17 Nov 2026, 12:00 CET	**Quotation deadline**
Wed 18 Nov	Decision communicated
Fri 20 Nov	**Kick-off call** (1 hour; Wed 25 Nov at the latest). The project team hands over what section 8 lists, and the versions to target are agreed (section 6).

DATE	STEP
Fri 15 Jan 2027	**Delivery, at the latest:** the pull requests ready for review with CI passing, the upgrade report and the verification results (section 9)
Fri 22 Jan	The project team sends one consolidated round of feedback
Fri 29 Jan	**Fixes delivered, acceptance confirmed by the project team, and invoice submitted**
February 2027	The team releases the upgraded app to the stores
Until Wed 31 Mar 2027	Bug fixes at no extra cost

The package is invoiced when it is accepted. Payment follows UPF's standard procedures after the invoice is accepted; UPF will tell the selected contractor what invoicing requirements apply. Please confirm in your quotation that you can meet these dates. They are latest dates: work delivered earlier is reviewed, accepted and invoiced earlier.

---

4 Scope

4.1 In scope

One package, awarded to one contractor:

- **Flutter and Dart** on the current stable release.
- **Every Dart package** at its current version, new major versions included, with the code changes the migrations need.
- **The Android build:** Gradle, the Android Gradle Plugin, Kotlin and the Google plugins current, the workarounds the old versions needed removed, and the app made as ready as its plugins allow for built-in Kotlin, the change that Flutter warns about on every Android build.
- **The iOS build:** the plugins' native dependencies in one consistent, locked state, building with the current Xcode.
- **CI and Dependabot:** the workflows passing on current action versions, and the open Dependabot pull requests dealt with.
- **Documentation** corrected, and an **upgrade report**.
- **Verification** on devices that nothing users rely on has changed, above all their account and their saved data.

4.2 Out of scope

- **The backend and the project's API SDK.** The `mosquito_alert` package, generated from the API description, stays at Git tag `0.4.1`. Newer tags exist (`0.4.15` at the time of writing) and change the API models; moving to them is separate work.
- **New features, redesign and refactoring**, and fixes for existing defects that the upgrade does not touch, such as the iOS launch screen (issue #798) or the Firebase configuration of the iOS test app (issue #786). If you find a defect, report it as an issue.
- **Changes to what ships beyond the upgrade itself**, such as switching on code shrinking, changing permissions or raising the minimum OS versions.
- **Translations and store listings.**

- **Store releases and store administration.** The team builds, signs and uploads releases and holds the signing keys, which the contractor never receives.

5 The app today

The repository is public and is the reference: read it before quoting. The facts below were checked on 9 November 2026. Appendix A has the version inventory and Appendix C the environments.

The base for the work is `main` at kick-off. Today `main` is one release behind. Version 4.4.0, which is in both stores, sits in pull request #788, and seven further pull requests with fixes (#792–#797 and #799) are open against it. The team merges #788 before kick-off, together with those of the seven that are ready. Two of them change dependencies: #794 adds Firebase Crashlytics, and #795 replaces `overlay_support` with `flutter_local_notifications`. From kick-off until delivery the team keeps `main` stable: nothing that touches dependencies or build files is merged without telling the contractor first.

ITEM	TODAY
Code	Flutter 3.44.1 (fvm, `.fvmrc`), Dart SDK constraint `>=3.12.1`; 137 Dart files, about 13,500 lines including generated code, organised by feature under `lib/features/` with shared code in `lib/core/`
State and data	`provider` for state; `dio` through the generated `mosquito_alert` SDK for the API; `hive_ce` for local storage and the queue of reports waiting to be sent; `flutter_secure_storage` for credentials; `shared_preferences` for settings; `workmanager` for periodic background tasks
Platform plugins	`camera`, `image_picker` with the system photo picker, `google_maps_flutter`, `geolocator`, `geocoding`, `permission_handler`, `firebase_core`, `firebase_messaging`, `firebase_analytics`, `webview_flutter`, `flutter_html`, `in_app_review`
Apps	Production: Android flavour `prod` (`ceab.movelab.tigatrapp`) and iOS scheme `Runner` (`cat.ibeji.tigatrapp`), entry point `lib/main.dart`. Test: flavour `dev` (`ceab.movelab.tigatrapp.test`) and scheme `devtf` (`com.mosquitoalert.devtest`), entry point `lib/main_dev.dart`. A start-up guard and a unit test keep each build paired with its backend (Appendix C)
Android build	Gradle 9.1.0, Android Gradle Plugin 9.0.1, Kotlin 2.3.20, Google Services plugin 4.4.4, Java 17; `compileSdk` 36, `targetSdk` 36 and `minSdk` 24, all from Flutter's defaults; workarounds and compatibility settings as listed in Appendix A.3; code shrinking off
iOS build	Deployment target 17.0; Xcode 26; Swift Package Manager for 22 of the 25 plugins and CocoaPods for the other three; six build configurations behind the two schemes
CI	GitHub Actions. On every pull request: analysis with unit and widget tests; an Android debug bundle and an unsigned iOS build; one integration test on an Android emulator. On version tags: two workflows that exchange data with the project's translation platform. Dependabot opens pull requests monthly for pub, Gradle and Actions; nine are open (#760–#766, #772, #780)
Tests	47 unit and widget tests in eight files, with mocks under `test/mocks/`; one integration test; the analyser reports no issues
Release	Version 4.4.0 (build 2939), in the App Store since September 2026 and on Google Play since October 2026. Releases are built, signed and uploaded by the team
Documentation	`README.md` (set-up for both platforms), `docs/` (store listings, translations), `.github/copilot-instructions.md` (out of date: it still names Flutter 3.27 and builds without flavours)

What the app keeps on the device, and why it is the main risk. Users are never asked for an email address or a password. On first start the app registers a guest account by itself and keeps the credentials on the device. If they are lost, the user loses the account with its history and its score, and the app offers no way to recover it. Three stores are involved:

- `flutter_secure_storage` holds the account's username and password, the access and refresh tokens, the device identifier, and the key that encrypts one box of the local database.
- `hive_ce` holds the local database: the account record (encrypted with that key), copies of the user's reports, location fixes, and the queue of reports waiting to be sent.
- `shared_preferences` holds the settings and a first-run flag. When that flag is missing, the app takes the start for a fresh install and **erases the secure storage** (`clearSecureStorageOnReinstall()` in `lib/main.dart`).

All three packages are upgraded in this work, `flutter_secure_storage` by a major version. A change in where or how any of them stores its data would, without a migration, sign users out of accounts that cannot be recovered. Requirements KEEP-02 and VER-03 exist for this.

---

6 Requirements

This section says what must be true at the end. The order of the upgrades, the tools and the number of pull requests are yours to decide. The appendices report what we found in a short trial, including two problems in the Android build. They are there to save you time, not to prescribe the fix: where your experience points to a better route, take it.

Current means the latest stable version that works with the Flutter release agreed at kick-off, as published on the kick-off date. `flutter pub outdated` calls it the *resolvable* version. Pre-releases do not count, and versions published during the work are optional.

The Flutter release agreed at kick-off is the latest stable one on that date: 3.47.9 today. Flutter 3.50 is in beta. If it becomes stable shortly before kick-off, we decide together at kick-off whether to target it or the last 3.47 release.

6.1 Up to date (UPG)

ID	PRIORITY	REQUIREMENT
`UPG-01`	`MUST`	**Flutter and Dart.** The app builds and runs on the Flutter release agreed at kick-off, pinned in `.fvmrc` , which fvm reads locally and CI reads too. The Dart SDK constraint in `pubspec.yaml` follows it. Analyser issues and deprecations that the new versions raise in the app's own code are fixed, not suppressed. A deprecation whose replacement is not yet usable, or whose fix reaches well beyond the upgrade, may be left and is listed in the upgrade report.
`UPG-02`	`MUST`	**Packages.** Every direct dependency and dev dependency in `pubspec.yaml` is at its current version, new major versions included, with the code changes each migration needs. At delivery, `flutter pub outdated` shows no direct dependency below its resolvable version other than the hold-backs of UPG-03.

ID	PRIORITY	REQUIREMENT
UPG-03	MUST	**Hold-backs are few, reasoned and agreed.** A package stays behind only for a concrete reason: this request fixes its version (the `mosquito_alert` SDK stays at tag `0.4.1`, and `built_collection` at 5.x because the SDK requires it); the newer version breaks another package that has no fixed release (Appendix A.5 has an example); it has a defect that the contractor has reproduced; or adopting it needs a decision that is the team's (TECH-05). Each hold-back is agreed with the team before delivery and recorded in the upgrade report with its reason and the upstream reference.
UPG-04	MUST	**Android build.** Gradle, the Android Gradle Plugin, the Kotlin Gradle Plugin and the Google Services plugin, and any Gradle plugin added by a pull request merged before kick-off, are at a combination of current versions that builds both apps with the agreed Flutter release and the upgraded packages. The contractor chooses the combination and gives the reasons in the upgrade report. Our trial met one conflict on the way and found one combination that builds (Appendix A.5); there may be better ones. `compileSdk` and `targetSdk` satisfy the Google Play requirement in force at delivery, and the Java version of the build and of CI is the one the toolchain needs.
UPG-05	MUST	**The Kotlin Gradle plugin warning.** Every Android build of the app prints a Flutter warning that names plugins which apply the Kotlin Gradle plugin themselves, and says that a future Flutter version will fail to build them (Appendix A.4). After this work the app is as ready for that change as its plugins allow. How to get there, and how to show it, is for the contractor to propose. The upgrade report says which plugins the warning still names and why, and what the team should do about any that would break the build. The two flags that Flutter's own tooling writes to `android/gradle.properties` stay as that tooling leaves them.
UPG-06	MUST	**Workarounds.** Each workaround and compatibility setting in Appendix A.3 is removed where the upgraded plugins no longer need it, and otherwise kept with a comment naming the upstream change that will allow its removal.
UPG-07	MUST	**iOS dependencies.** Plugins that ship a Swift package at their current version are consumed through Swift Package Manager. CocoaPods stays only for the plugins that still need it, and is removed from the project and from the documentation if none does. What remains is locked in the repository (`Package.resolved`, and `Podfile.lock`, which Git ignores today), so that two clean clones build the same versions. Both schemes build from a clean clone.
UPG-08	MUST	**CI.** The three workflows that run on pull requests pass on the delivered pull requests, with their actions at current major versions and their runner image, Xcode, Java and emulator settings matching the new toolchain. The two translation workflows receive the same updates but are **not run** by the contractor: they write to the project's translation platform. Each Dependabot pull request open at kick-off is either covered by the work or declined for a reason given in the upgrade report, so that the team can close them all.
UPG-09	SHOULD	**Dependabot.** `.github/dependabot.yml` groups related updates (the Firebase packages together, for example), so that future updates arrive as a few pull requests that can be checked together.
UPG-10	SHOULD	**Build warnings.** Warnings from the Android and iOS builds that the project itself can fix are fixed (Gradle deprecations, project settings that Xcode offers to update). Those that wait for an upstream release are listed in the upgrade report.
UPG-11	MUST	**Documentation.** `README.md`, `.github/copilot-instructions.md` and the files under `docs/` are corrected wherever a version, a command or a step has changed. A developer who follows the README builds and runs the test app on both platforms from a clean clone.

6.2 Unchanged for users (KEEP)

ID	PRIORITY	REQUIREMENT
KEEP-01	MUST	**No functional change.** Screens, texts, behaviour and the format of stored data stay as they are, except where an upgrade forces a change. Every forced change is listed in its pull request and in the upgrade report.
KEEP-02	MUST	**Users keep their account and their data.** A device on which version 4.4.0 is installed and in use, and which updates in place to the upgraded build, keeps its account and stays signed in, keeps its settings (language, background tracking), keeps the reports saved on the device, and sends those that were waiting at the next sync. The onboarding and consent screens do not appear again. Where a new version changes where or how a package stores its data, the migration is part of the work, and the migrations the app already runs on data left by older versions stay in place. Section 5 says what is stored where.
KEEP-03	MUST	**Supported systems.** Android 7.0 (API 24) and iOS 17.0 remain the minimum versions.
KEEP-04	MUST	**Permissions and store declarations.** The upgrade report compares, before and after, the permissions in the merged Android manifest of the release builds and, on iOS, the usage descriptions, background modes and entitlements. Any difference is reported to the team and approved before delivery. The app deliberately declares no photo or media permission, to comply with Google Play's photo and video permissions policy, and an upgraded plugin must not bring one back.
KEEP-05	MUST	**Both apps.** The production and test apps build on both platforms (Appendix C), and the guard that pairs each build with its backend stays in place and passes: `AppConfig.assertMatchesPackage()` and `test/unit/environment_config_test.dart`.
KEEP-06	MUST	**Size.** The upgrade report gives the size of the Android App Bundle and of the iOS app before and after (version 4.4.0: an 86 MB bundle, and 59 MB on the App Store), and explains any growth of more than 10%.

6.3 Verified (VER)

ID	PRIORITY	REQUIREMENT
VER-01	MUST	**Automated checks.** `flutter analyze` reports no issues, the code is formatted with `dart format`, and the unit, widget and integration tests pass in CI. Tests are adapted where an API changed; none is removed or skipped.
VER-02	MUST	**Devices.** The checklist of Appendix B passes on release-mode builds of the test app on at least two Android phones, one on Android 15 or later and one on Android 10 or earlier (an emulator with Google Play services will do for the older one), and on one physical iPhone on the current iOS. The app also starts and completes a report on emulators at the minimum versions, Android 7.0 and iOS 17. The delivery includes the checklist filled in for each device. One item cannot be checked on the iOS test app today: push notifications, because that app still uses the production app's Firebase configuration (issue #786). The team checks it on a TestFlight build of the production app, on request and at least once before delivery.
VER-03	MUST	**Update in place.** KEEP-02 is demonstrated on one Android phone and on iOS (a physical iPhone through the team's TestFlight uploads, or the simulator). With the test app as it is at kick-off installed (built from the kick-off commit, or taken from the test tracks), let it register, change the language, switch background tracking on and save a report while offline. Then install the upgraded build over it and check each point of KEEP-02, including after the device has been restarted. The delivery includes the steps taken and the result on each platform.

ID	PRIORITY	REQUIREMENT
VER-04	MUST	**Release builds.** `flutter build appbundle --release` for both flavours and `flutter build ipa` for both schemes succeed from a clean clone by following the README, with the contractor's own signing on Android and unsigned on iOS. The team repeats them with its own signing at acceptance.

7 Working rules (TECH)

ID	PRIORITY	REQUIREMENT
TECH-01	MUST	**Pull requests.** The work is delivered as pull requests against `main` of the public repository, from branches named in the repository's style (`chore/<topic>`), with commit messages in its prefix style (`chore:` , `fix:` , `ci:` , `docs:` , `test:`) and a description of what changed and how it was tested. A first pull request is opened as a draft early in the work, so that CI runs on it and the team can follow. The team reviews and merges.
TECH-02	MUST	**A history that can be reviewed and undone.** The Flutter and build-tool changes, the routine package updates, and each major-version migration that needed code changes are separate pull requests, or separate commits within one, that each build and pass the tests. A regression can then be traced to one step, and that step reverted.
TECH-03	MUST	**Base.** The work starts from `main` at kick-off and is rebased on the current `main` before delivery. The team keeps `main` stable in between (section 5).
TECH-04	MUST	**Test app and test backend only.** Development and testing use the test app against the test backend. The contractor builds the production app but does not use it, and sends nothing to the production backend.
TECH-05	MUST	**Decisions that are the team's.** The contractor raises these as soon as they come up, and the team answers within one working day: raising a minimum OS version; adding, removing or changing a permission, an entitlement or a store declaration; replacing or removing a package, or adding a new direct one; holding a package back (UPG-03); and anything that changes what users see or what data the app collects or stores.
TECH-06	MUST	**Secrets.** Nothing that `.gitignore` excludes (configuration files, keys, keystores) is committed, pasted into issues, pull requests or CI logs, or shared outside the project. The repository and its CI logs are public.
TECH-07	MUST	**Licence.** The app is GPL-3.0 and the work is contributed under that licence. A package that replaces another, or is added, must have a compatible licence and need no paid service.

8 What the project team provides

At the kick-off on **Friday 20 November 2026**, unless noted:

1. **Repository access.** Write access to the public repository for branches and pull requests. CI needs the repository's secrets, which pull requests from forks do not receive.
2. **A stable base.** `main` with release 4.4.0 merged, and no changes to dependencies or build files during the work without notice (section 5).
3. **Build configuration for the test app.** The Android `google-services.json` (one file covers both flavours), the iOS `GoogleService-Info.plist` , and a Google Maps key for the Android test app authorised for the contractor's signing certificate (send its SHA-1 fingerprint at kick-off). None of these

may be committed (TECH-06). The build also reads a `.env` file, for which the placeholder that CI writes is enough.

4. **The test backend**, `apidev.mosquitoalert.com`. The test app registers its own accounts there, and test reports can be created freely. On request the team confirms what reached the backend: reports, location fixes, push tokens.
5. **Test builds signed by the team.** Tester access to *Test Mosquito Alert* on TestFlight and on Play internal testing. On request, and within one working day, the team builds the contractor's branch and uploads it to those tracks, up to three times per platform. This is how the upgraded app reaches a physical iPhone.
6. **Push notifications** sent to the test app on request, and the iOS push check on the production app (VER-02), each within one working day.
7. **One contact person**, John Palmer, who answers questions and takes the decisions of TECH-05 within one working day, and reviews the delivery within five working days. Between 24 December and 6 January, answers and uploads may take longer.

---

9 Deliverables and acceptance

9.1 Deliverables

By **Friday 15 January 2027** at the latest:

1. **The pull requests** (TECH-01, TECH-02), ready for review, rebased on `main`, with CI passing.
2. **The upgrade report**, a Markdown file in `docs/`: every version before and after (toolchain and direct packages); each migration that needed code changes, and what changed; the hold-backs, with reasons and upstream references; the workarounds removed and kept; the plugins that the Kotlin Gradle plugin warning still names (UPG-05); the forced functional changes (KEEP-01); the comparison of permissions and declarations (KEEP-04); the sizes (KEEP-06); the warnings that remain; recommended follow-ups; and a short procedure for repeating the sweep next time.
3. **The verification results:** the Appendix B checklist filled in for each device (VER-02), and the update-in-place steps and results (VER-03).
4. **The corrected documentation** (UPG-11).

After acceptance: a 30-minute handover call, and bug fixes at no extra cost until Wednesday 31 March 2027. The team plans the first store release from the upgraded code for February 2027, and defects caused by the upgrade that appear in that release are covered.

9.2 Acceptance

The work is delivered when the four items above are in. The project team reviews it and sends one consolidated round of feedback within five working days, by Friday 22 January at the latest. The contractor delivers the fixes within five working days of the feedback. The work is accepted when:

1. Every **MUST** requirement is met, together with the SHOULD requirements that the quotation included.
2. All CI checks pass on the final pull requests, and the team builds signed release builds of both apps, on both platforms, from the branch on its own machines by following the README.
3. The team's own checks pass on its devices: the Appendix B checklist; push notifications on iOS; and an update in place from the 4.4.0 production app installed from the stores to a team-signed build of the upgraded code, with the account and the data kept (KEEP-02).

4. `flutter pub outdated` shows no direct dependency below its resolvable version except the agreed hold-backs, each item of Appendix A.3 is gone or justified again, and the upgrade report accounts for every plugin that the Kotlin Gradle plugin warning still names (UPG-05).

---

10 How to submit a quotation

Send your quotation by email to **john.palmer@upf.edu**, copying **it@mosquitoalert.com**, by **Tuesday 17 November 2026, 12:00 CET**. A short quotation is enough. Please include:

1. **A fixed price excluding VAT** for the package, with the applicable VAT shown separately (if you are established outside Spain, say whether the reverse charge applies instead). Tell us which SHOULD items you would drop to reduce the price, and by how much.
2. **Confirmation that you can deliver by 15 January 2027.** Earlier delivery is welcome.
3. **2–3 examples of relevant work**, such as Flutter apps in the stores that you have built or maintained, with links. If you have worked on the Mosquito Alert app before, it is enough to say so.
4. **Assumptions and exclusions.**

We will choose based on relevant experience, price (compared excluding VAT), and ability to meet the dates. UPF may decide not to award the work.

A Versions today

Checked on 9 November 2026 against the 4.4.0 code (pull request #788). This appendix is information for quoting: the versions to reach are fixed at kick-off (section 6).

A.1 Toolchain

COMPONENT	IN USE	CURRENT, AND WHAT FLUTTER 3.47.9 EXPECTS
Flutter	3.44.1 (`.fvmrc` , read by fvm locally and by CI)	3.47.9, released 29 October 2026; 3.50 is in beta
Dart	SDK constraint `>=3.12.1 <4.0.0`	Dart 3.13.7 ships with Flutter 3.47.9
Gradle wrapper	9.1.0	Flutter 3.47.9 creates new projects with 9.3.1, the newest it lists as supported. Dependabot proposes 9.6.1 (#766), which passes CI on the current code
Android Gradle Plugin	9.0.1	New projects use 9.1.0, which recommends compile SDK 36 at most. Flutter 3.47.9 knows versions up to 9.2; 9.2.0 needs Gradle 9.4.1 or later
Kotlin Gradle Plugin	2.3.20	2.4.0
Google Services plugin	4.4.4	4.5.0 (Dependabot #763)

COMPONENT	IN USE	CURRENT, AND WHAT FLUTTER 3.47.9 EXPECTS
`compileSdk`, `targetSdk`, `minSdk`, NDK	36, 36, 24 and NDK 28.2, all Flutter defaults	The same defaults in Flutter 3.47.9. `permission_handler` 13 compiles against SDK 37 (Appendix A.5)
Java	17 for source and target; Temurin 17 in CI	17 is the minimum for Flutter 3.47.9
iOS deployment target	17.0	Stays (KEEP-03)
Xcode	26.2 on the `macos-15` runner in CI; the README asks for 26.1.1 or later	
iOS plugin dependencies	Swift Package Manager for 22 of 25 plugins. CocoaPods for `flutter_image_compress_common`, `google_maps_flutter_ios` and `workmanager_apple`. `Package.resolved` is committed; `Podfile.lock` is ignored by Git	Of the three, only `google_maps_flutter_ios` still ships no Swift package at its current version
GitHub Actions	`actions/checkout@v6`, `actions/setup-java@v5`, `actions/setup-python@v6`, `codecov/codecov-action@v6`, `kuhnroyal/flutter-fvm-config-action@v3`, `reactivecircus/android-emulator-runner@v2`, `peter-evans/create-pull-request@v8`, `fjogeleit/http-request-action@v2`	Dependabot proposes `checkout` 7 (#764), `codecov-action` 7 (#765), `setup-python` 7 (#772) and `setup-java` 6 (#780)
Dependabot	Monthly, ungrouped: pub (at most 3 open pull requests), Gradle (at most 2), GitHub Actions	

A.2 Direct packages behind their latest version

From `flutter pub outdated` on 9 November 2026. The app has 55 direct packages from pub.dev (48 runtime and 7 dev), besides those of the Flutter SDK and the project's API SDK. 16 are current and 39 are behind, ten of them by a new major version:

PACKAGE	IN USE	LATEST	WHERE THE APP USES IT
`badges`	3.2.0	4.0.1	The badge on the notifications icon
`built_collection`	5.1.1	6.0.0	**Cannot be adopted:** the project's SDK requires 5.x
`cached_network_image`	3.4.1	4.0.6	Report photos
`firebase_analytics`	12.4.3	13.0.1	Screen and event logging, throughout the app
`firebase_core`	4.11.0	5.1.0	Start-up
`firebase_messaging`	16.4.1	17.0.1	Push token registration and incoming notifications
`flutter_secure_storage`	10.3.1	11.2.0	Credentials and the database key (section 5)
`geocoding`	4.0.0	5.0.1	Place names of reports saved on the device; locale set-up
`permission_handler`	12.0.3	13.0.2	Camera and background-tracking permissions
`workmanager`	0.9.0+3	0.10.10	Periodic sending of waiting reports; background tracking

The other 29 are minor or patch updates. Runtime (25): app_set_id 1.4.0 → 1.6.0; battery_plus 7.1.0 → 7.1.2; built_value 8.12.6 → 8.13.0; camera 0.12.0+1 → 0.12.2; connectivity_plus 7.2.0 → 7.3.2; country_codes_plus

5.1.1 → 5.2.1; device_info_plus 13.2.0 → 13.3.0; dio 5.9.2 → 5.11.1; envied 1.3.7 → 1.3.10; flutter_image_compress 2.4.0 → 2.5.1; geolocator 14.0.3 → 14.1.2; google_maps_flutter 2.17.1 → 2.19.0; hive_ce 2.19.3 → 2.23.2; hive_ce_flutter 2.3.4 → 2.5.0; html 0.15.6 → 0.15.7; image_picker 1.2.2 → 1.2.4; image_picker_android 0.8.13+17 → 0.8.13+25; image_picker_platform_interface 2.11.1 → 2.11.2; internet_connection_checker_plus 3.1.0 → 3.1.2; intl 0.20.2 → 0.20.3; package_info_plus 10.2.0 → 10.2.2; shared_preferences 2.5.5 → 2.5.6; synchronized 3.4.0+1 → 3.6.1+1; uuid 4.5.3 → 4.6.0; webview_flutter 4.14.0 → 4.14.1. Dev (4): build_runner 2.15.0 → 2.17.1; envied_generator 1.3.7 → 1.3.10; hive_ce_generator 1.11.2 → 1.12.1; json_serializable 6.14.0 → 6.15.2.

Not to be changed: `mosquito_alert` (Git tag `0.4.1`), the project's API SDK. Three of the packages above have Dependabot pull requests open: `dio` (#762), `internet_connection_checker_plus` (#761) and `synchronized` (#760).

A.3 Workarounds and compatibility settings in the Android build

WHERE	WHAT	WHY IT IS THERE
`android/build.gradle.kts`	`androidx.work:work-runtime` forced to 2.11.2 in every configuration	Added for `workmanager` 0.9, with the note "remove when flutter_workmanager has been updated"
`android/build.gradle.kts`	A `compileOnly` dependency on `androidx.concurrent:concurrent-futures` added to the `camera_android_camerax` module	Version 0.7.2 of that plugin does not compile with the JDK bundled with current Android Studio; the comment says to remove it once the plugin is past 0.7.2
`android/gradle.properties`	`android.newDsl=false` and `android.builtInKotlin=false`	Not this project's workaround: Flutter's own tooling writes these two flags for projects on Android Gradle Plugin 9, and still does in Flutter 3.47.9 and in the 3.50 beta. They stay as that tooling leaves them (UPG-05, Appendix A.4)
`android/gradle.properties`	`android.enableJetifier=true`	The comment says `permission_handler` needs it, which is probably no longer true

A.4 The Kotlin Gradle plugin warning

Every Android build of the app prints this warning from Flutter: "Your app uses the following plugins that apply Kotlin Gradle Plugin (KGP) [...] Future versions of Flutter will fail to build if your app uses plugins that apply KGP." It is the open end of the move to Android Gradle Plugin 9, which compiles Kotlin itself, and every Flutter app is going through it. The warning links to Flutter's own migration guides. The app's build files were migrated by that guide in June 2026 (pull request #743). What remains is in the plugins, and for them the guidance is to upgrade and, where no migrated release exists, to wait for one.

- **Today the warning names twelve plugins:** `app_set_id`, `camera_android_camerax`, `country_codes_plus`, `firebase_analytics`, `flutter_image_compress_common`, `google_maps_flutter_android`, `image_picker_android`, `in_app_review`, `shared_preferences_android`, `url_launcher_android`, `webview_flutter_android` and `workmanager_android`.

- **At their current versions, eight of them no longer apply the Kotlin plugin.** Three apply it only while built-in Kotlin is switched off: `firebase_analytics`, `flutter_image_compress_common` and `workmanager_android`, and so does `firebase_core` 5. Flutter 3.47.9 finds the plugin by scanning build files, so it goes on naming these four, together with `in_app_review` (Appendix A.5).
- **One has no release that supports built-in Kotlin:** `in_app_review` 2.0.12, its latest (May 2026), applies the Kotlin plugin unconditionally. Its repository has several open issues and unmerged pull requests for exactly this. The app uses it for the store review prompt. With built-in Kotlin switched on, the build stops there.
- **There is no deadline yet.** Flutter 3.47.9 and the 3.50 beta only warn, and both still write the two flags of Appendix A.3 into the project.
- **Two earlier updates in this area were closed after the integration tests failed in CI:** the Android Gradle Plugin to 9.2 (#724, before the app's own migration) and Kotlin to 2.4.0 (#750). The cause was not recorded. The move to Flutter 3.44 also needed a later fix for the Android release build (#769).

A.5 Trial upgrade, 9 November 2026

To size the work, we tried the upgrade in a scratch copy of the 4.4.0 code: the Dart side first, then the Android build. It was a short trial and is not exhaustive. This is information, not a prescription of the approach.

1. **Flutter 3.47.9 with the packages unchanged.** Dependencies resolve. The analyser reports one warning (`unawaited_return_in_try_block`, in `auth_repository.dart`). The 47 unit and widget tests pass.
2. **Then every package to its latest resolvable version** (`flutter pub upgrade --major-versions`). Nine new major versions resolve: `badges` 4, `cached_network_image` 4, `firebase_analytics` 13, `firebase_core` 5, `firebase_messaging` 17, `flutter_secure_storage` 11, `geocoding` 5, `permission_handler` 13 and `workmanager` 0.10. `built_collection` stays at 5.x, as expected. The analyser reports two errors, both from `geocoding` 5, whose top-level functions are gone (two call sites), and two deprecations from `firebase_messaging` 17 (`getToken` and `onTokenRefresh`).
3. **One incompatibility between packages.** After step 2 the app no longer compiles, because `flutter_html` 3.0.0, its latest release (March 2025), calls an internal function that `html` 0.15.7 removed. Holding `html` at 0.15.6 is the obvious way out; replacing `flutter_html` would be a decision for the team (TECH-05).
4. **The Android build then fails with the build tooling in use.** `permission_handler` 13 compiles against Android SDK 37, which Android Gradle Plugin 9.0.1 cannot use. With Android Gradle Plugin 9.1.0, Gradle 9.3.1 and Kotlin 2.4.0 the plugins compile, but the build then requires the app itself to compile against SDK 37, and Android Gradle Plugin 9.1.0 recommends 36 at most.
5. **One combination that builds.** With `compileSdk` 37, Android Gradle Plugin 9.2.0, Gradle 9.4.1, Kotlin 2.4.0 and Google Services 4.5.0, and without the first two workarounds of Appendix A.3, the debug build and the release bundle of the test app both build. For this step `geocoding` and `html` stayed at their present versions, so that the Dart code compiled unchanged. Gradle 9.4.1 is newer than Flutter 3.47.9 lists as supported. We did not try `compileSdk` 37 with Android Gradle Plugin 9.1.0 and Gradle 9.3.1, the versions Flutter 3.47.9 uses itself, which may also build. Holding `permission_handler` at 12 avoids the question.
6. **Built-in Kotlin.** In that build Flutter's warning still names five plugins: `firebase_analytics`, `firebase_core`, `flutter_image_compress_common`, `in_app_review` and `workmanager_android`. With built-in Kotlin switched on, the build stops at `in_app_review`. When the two flags of Appendix A.3 are deleted, Flutter's tooling writes them back.

7. **Not tried:** the iOS build, CI, and anything at run time. A build that succeeds says nothing about behaviour. The major versions may still change it, in particular `flutter_secure_storage` 11, `workmanager` 0.10, `permission_handler` 13 and the Firebase packages. Section 6.3 is there for that.

B What to check on each device

The checklist for VER-02, run on release-mode builds of the test app. A note in italics names the upgraded package that an item exercises most.

1. **Fresh install.** The onboarding and consent screens appear once. The language follows the device and can be changed in Settings; after choosing Arabic, the layout is mirrored.
2. **Permissions.** Location (while in use, and the background option), camera and notifications: grant and deny each, and check the app's behaviour afterwards (explanations, links to the system settings, no crash). *`permission_handler` 13.*
3. **Reports.** A bite report; a mosquito report with one photo taken with the camera and one chosen with the system photo picker; a breeding-site report. The location step with an automatic fix and with a point chosen on the map, whose tiles load. Submission online, and submission in flight mode followed by a sync when the connection returns. Reports show their place name in the list and in the detail, including one saved offline. *`geocoding` 5, `camera`, `image_picker`, `google_maps_flutter`.*
4. **My Reports.** The list, the detail of each report type with its photos, the pending and sent indicators, and deletion. *`cached_network_image` 4.*
5. **Public map** opens and loads. *`webview_flutter`.*
6. **Background work.** Background tracking can be switched on and off in Settings and, when on, location fixes reach the test backend. With the app closed and the screen locked, a report saved offline is sent by the periodic task after the connection returns. This is checked on Android. On iOS the system decides when background tasks run: leave the app installed for a day with a report waiting, and record whether it was sent. *`workmanager` 0.10.*
7. **Notifications.** A push notification sent by the team arrives with the app in the foreground, in the background and closed; tapping it opens the right screen; the in-app list shows it with its badge; and the device's push token reaches the test backend. On iOS, see the note in VER-02. *`firebase_core` 5, `firebase_messaging` 17, `badges` 4.*
8. **Settings and information pages.** Profile and score; the guide; the privacy, terms and licence pages in the selected language; the language picker. *`flutter_html`.*
9. **Session.** After the app is force-stopped, and after the device is restarted, the user is still signed in and the saved reports are still there. The update-in-place check of VER-03 is run separately. *`flutter_secure_storage` 11, `hive_ce`, `shared_preferences`.*
10. **Display.** Dark mode, and the system font size at 150%, on the main screens: nothing is cut off.

C Environments, identities and secrets

APP	IDENTITY	BUILT WITH	BACKEND
Mosquito Alert, Android	`ceab.movelab.tigatrapp`	Flavour `prod`, `lib/main.dart`	`api.mosquitoalert.com/v1`
Mosquito Alert, iOS	`cat.ibeji.tigatrapp`	Scheme `Runner`, `lib/main.dart`	The same

APP	IDENTITY	BUILT WITH	BACKEND
Test Mosquito Alert, Android	`ceab.movelab.tigatrapp.test`	Flavour `dev`, `lib/main_dev.dart`	`apidev.mosquitoalert.com/v1`
Test Mosquito Alert, iOS	`com.mosquitoalert.devtest`	Scheme `devtf`, `lib/main_dev.dart`	The same
Integration tests, Android emulator	The production identity	Flavour `prod`, environment `test`	None (a placeholder address)

- **Configuration.** Each entry point loads its environment from `assets/config/`: `prod.json`, `dev.json` or `test.json`.
- **Firebase.** One Firebase project holds the Android apps for both package names and the production iOS app. The Android `google-services.json` covers both flavours. The iOS test app has no Firebase app of its own yet and uses the production app's file (issue #786).
- **Google Maps.** Android has a separate key per flavour, injected at build time from `android/local.properties` or the environment; an empty key builds and shows a blank map. The iOS key is set in `AppDelegate.swift`. Keys are restricted to a package name and signing certificate, which is why the contractor receives a test-app key authorised for its own certificate.
- **Signing.** The team holds the Android keys (the production key is also the Play app-signing key; the test app has its own upload key) and the Apple Developer account. Without the team's `android/key.properties`, a release build needs a keystore of the contractor's own, set up in the same file; the build file shows the property names.
- **Kept out of Git** by `.gitignore`: keystores, `android/key.properties`, `android/local.properties`, `android/app/google-services.json`, `ios/Runner/GoogleService-Info.plist`, `.env`, and the translation-platform keys.

Mosquito Alert is coordinated by CEAB-CSIC, UPF, CREAF and ICREA. This specification is published under CC BY 4.0; the app code is GPL-3.0.

Mosquito Alert · www.mosquitoalert.com · github.com/Mosquito-Alert/mosquito_alert_mobile_app